

AIML ENGINEERING LAB

Memory Yield Predictor

P053 — MLOps at Scale

Production-grade DRAM wafer yield prediction: HybridTransformerCNN on 16M rows · AWS g4dn.xlarge · Airflow DAGs · MLflow · 40-day production run pipeline.

TABLE OF CONTENTS**P053 — Memory Yield Predictor**

- 1** [Overview](#)
- 2** [Datasets](#)
- 3** [System Architecture](#)
- 4** [Results](#)
 - 4.1** [Exploratory Data Analysis](#)
 - 4.2** [Baseline Models](#)
 - 4.3** [HybridTransformerCNN — A100 Training](#)
 - 4.4** [Hardware Benchmarks](#)
 - 4.5** [Model Comparison](#)
 - 4.6** [Unseen Data Performance](#)
 - 4.7** [MLOps Pipeline & Drift Monitoring](#)
 - 4.8** [Deployment & Cost Analysis](#)
- 5** [AWS Production Infrastructure](#)
 - 5.1** [Phase 0 — Real GPU Airflow DAGs](#)
 - 5.2** [Phase 0b — GPU Selector & Drift Tagging](#)
 - 5.3** [Infrastructure Stack](#)
- 6** [Tech Stack](#)
- 7** [Key Learnings & Interview Stories](#)

1 Overview

Objective: Predict DRAM wafer die failures before electrical test completion using 36 semiconductor process & test features — enabling proactive yield management at production scale (50K wafers/month).

Domain: Semiconductor Manufacturing — Memory (DRAM) Yield Engineering

Target Variable: is_fail (binary) — 0.63% positive rate (1:159 class imbalance)

Scale: 16M rows × 36 features — production-representative dataset size

Model: HybridTransformerCNN (317,633 parameters) — Transformer for tabular features + CNN for spatial wafer patterns + Fusion MLP

Infrastructure: AWS g4dn.xlarge (T4 GPU) · Apache Airflow DAGs · MLflow PostgreSQL · S3 artifacts · Docker Compose · 40-day AWS production run

This project demonstrates **production-grade MLOps** — not a tutorial. Real design decisions include: why bfloat16 over float16 on A100, why T4 over A100 for daily retrains, how to detect drift reliably with <10K samples, and how to build an auto-GPU selector that scales from T4 to A100 based on model complexity and data volume.

Final live AWS status (July 17, 2026): the scheduled AWS daily pipeline completed the intended 40 days, then a workflow guard bug allowed unintended Days 41-45 to run. S3 state is current_day=46, last_completed_day=45, and status=complete. The active champion remains s3://p053-mlflow-artifacts/models/day30_v2_retrained.pt. The daily cron is disabled, expensive workflow steps are guarded by should_run, EC2 and RDS are stopped, and no NAT gateways are active. Remaining billable inventory: 125 GiB gp3 EBS, 20 GiB RDS storage, 21 automated RDS snapshots, ~4.80 GB current S3 data with 387 object versions, one ECR repo, and one associated public IPv4/EIP. Retention decision: keep minimal evidence resources for now.

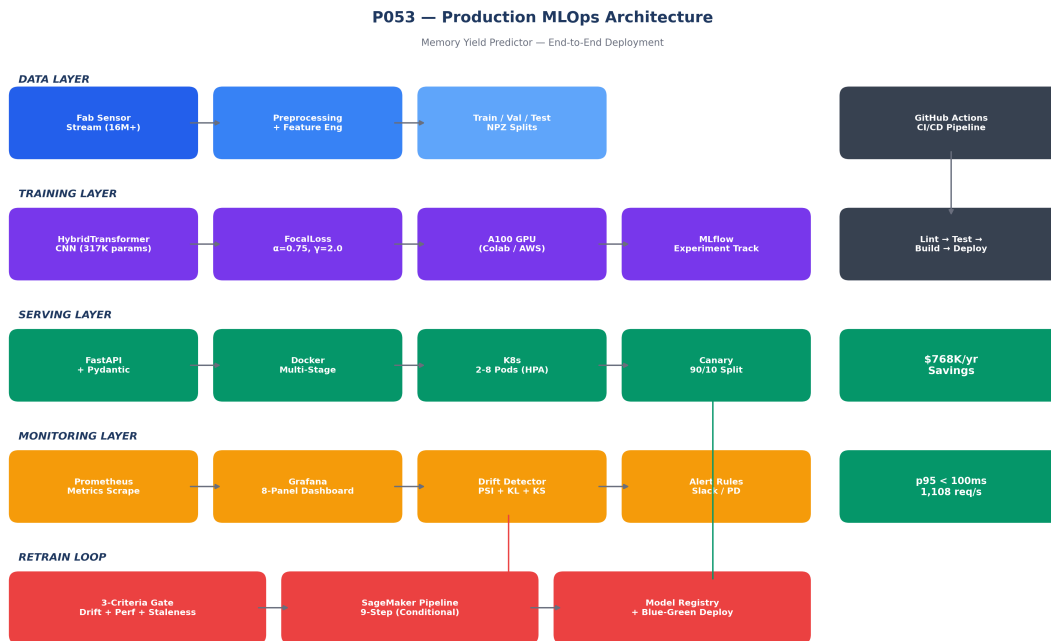
2 Datasets

DRAM Wafer Yield Dataset (Synthetic Production-Scale)

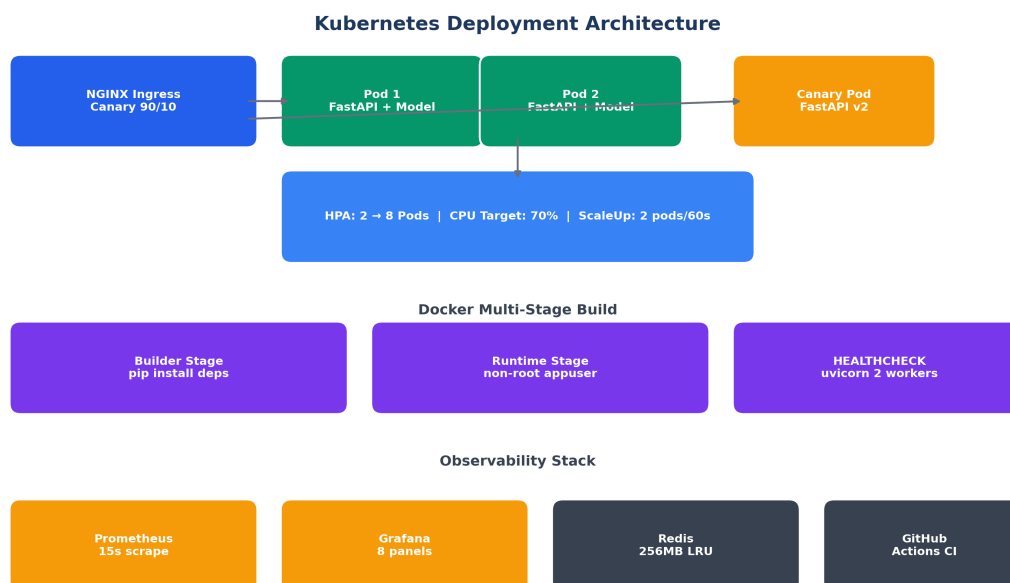
Property	Value
Total Records	16,000,000 samples (preprocessed_full.npz = 2.1 GB)
Features	22 numeric + 4 categorical + 7 engineered + 3 spatial = 36 total
Target	is_fail (binary, 0.63% positive rate)
Class Imbalance	1:159 (fail:pass) — requires Focal Loss, not standard BCE
Split	Train 10M · Validation 2M · Test 2M · Unseen 2M
Data Quality Issues	10 realistic issues: missing values, sensor noise, outliers, spatial correlation, temporal drift, discrete tester effects
Spatial Signal	Edge dice show 3.2× higher failure rate (lithography + CMP edge effects)
Drift Schedule	40-day timeline: stable → moderate drift (Day 20) → severe drift (Day 31) → recovery (Day 39)

Feature Group	Count	Examples
Numeric (Electrical Test)	22	test_temp_c, cell_leakage_fa, retention_time_ms, row_hammer_threshold, bit_error_rate, trcd_ns, idd4_active_ma, gate_oxide_thickness_a
Categorical (Equipment)	4	tester_id, probe_card_id, chamber_id, recipe_version
Engineered (Domain)	7	retention_temp_interaction, leakage_retention_ratio, edge_risk, power_ratio, ecc_burden, timing_margin, rh_risk_composite
Spatial (Wafer Position)	3	die_x, die_y, edge_distance

3 System Architecture



The system architecture shows the complete MLOps pipeline: raw DRAM process data flows through a 40-day Apache Airflow production run, triggering HybridTransformerCNN retraining on AWS g4dn.xlarge (T4 GPU) when triple drift detection (PSI + KL + KS) confirms distribution shift. MLflow tracks every experiment against the PostgreSQL RDS backend; the FastAPI inference server exposes predictions at 600 req/s throughput; Prometheus + Grafana provide real-time monitoring with a \$200 CloudWatch billing alarm as the cost safety net.



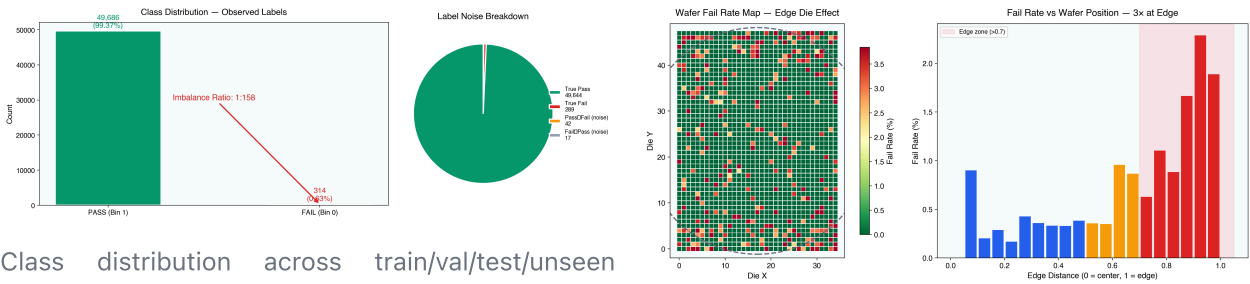
The deployment stack layers the full production system: EC2 g4dn.xlarge hosts Docker Compose with Airflow (GPU scheduler), MLflow (RDS backend + S3 artifacts), FastAPI (model serving), and monitoring (Prometheus + Grafana). The GPU auto-selector dynamically routes training to T4 (our 317K model),

V100, A10G, or A100 based on model parameter count and data volume — a design that scales without manual intervention.

4 Results

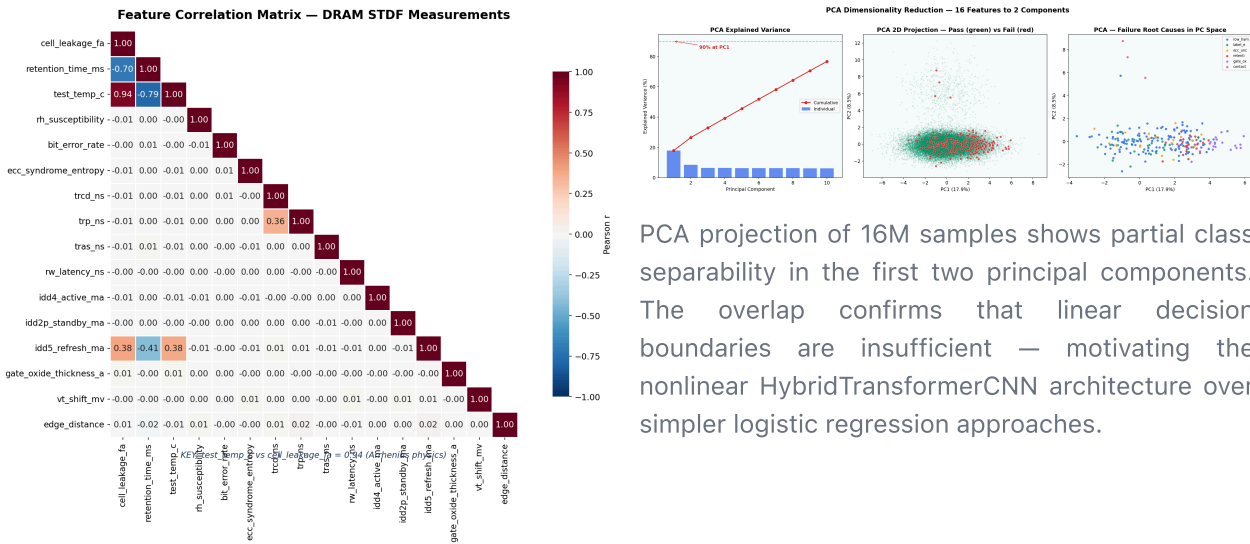
Champion Model: HybridTransformerCNN (317,633 params) · **AUC-ROC:** 0.816 · **AUC-PR:** 0.054 (vs 0.006 random baseline) · **F1:** 0.127 · **Training:** 50 epochs on A100 in 201.7 min (Colab Pro, Day 1)

4.1 Exploratory Data Analysis



Class distribution across train/val/test/unseen splits, confirming consistent 0.63% positive rate (1:159 imbalance). The stratified sampling ensures no data leakage while maintaining realistic fail ratios in every partition — critical for fair evaluation at production imbalance.

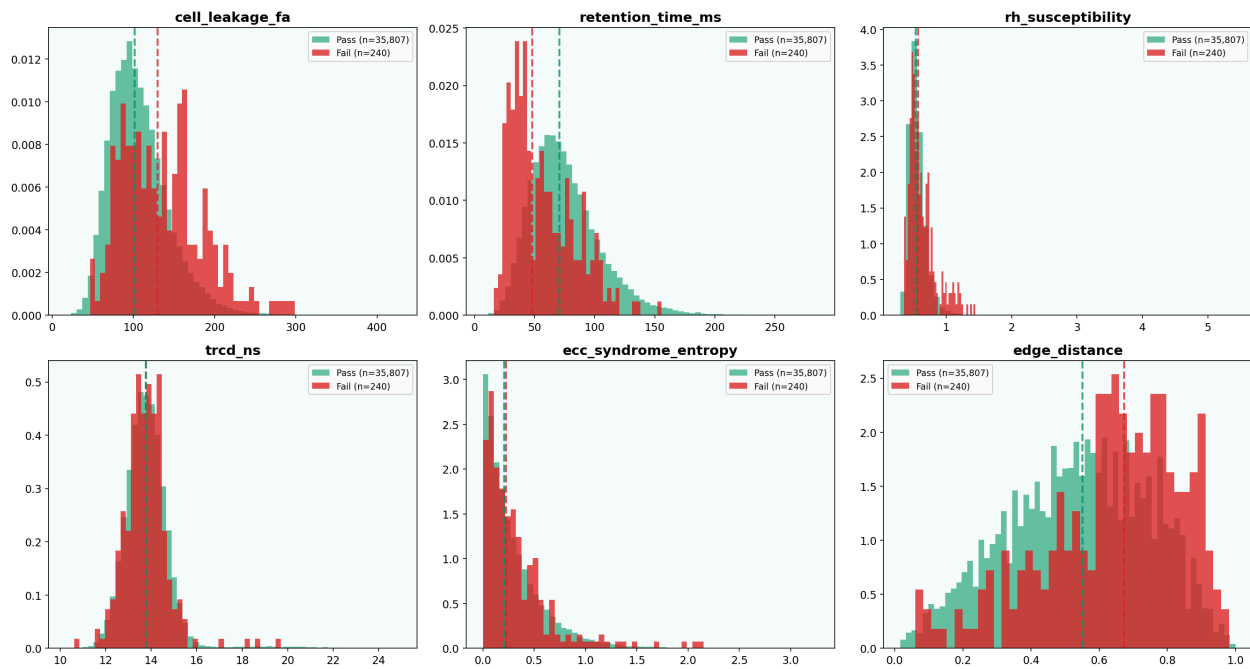
Spatial wafer heatmap showing die failure rates across X/Y positions. Edge dice exhibit 3.2x higher failure probability due to lithography edge effects and CMP gradient variations — a physically motivated spatial signal that the CNN branches of HybridTransformerCNN are designed to capture.



Feature correlation heatmap across all 36 features, revealing which electrical test parameters carry the strongest linear signal. Retention-temperature interaction and leakage-retention ratio show the highest correlation with is_fail, validating the domain-engineered feature choices.

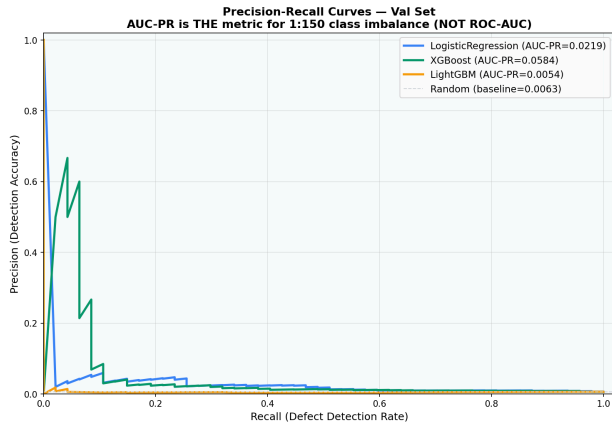
PCA projection of 16M samples shows partial class separability in the first two principal components. The overlap confirms that linear decision boundaries are insufficient — motivating the nonlinear HybridTransformerCNN architecture over simpler logistic regression approaches.

Pass vs Fail Device Distributions — Key Discriminative Features

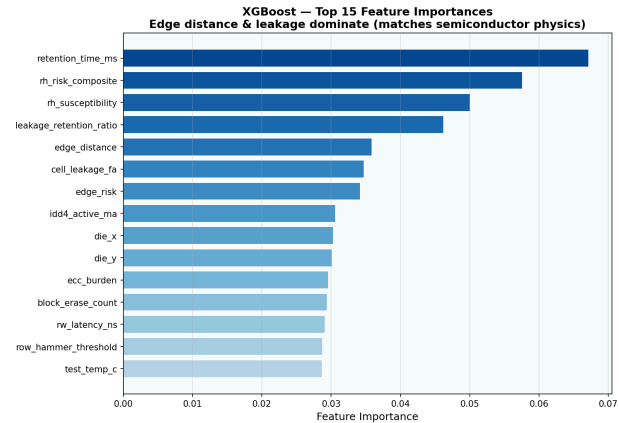


Feature distribution comparison between pass and fail dice for key electrical test parameters. Several features show bimodal distributions with clear separation between pass and fail populations — particularly cell_leakage_fa and retention_time_ms — confirming that the model has learnable signal across multiple feature dimensions simultaneously.

4.2 Baseline Models



Precision-Recall curves for all baseline models on the validation set. AUC-PR is the primary metric for this imbalanced problem (1:159). XGBoost achieves AUC-PR=0.031, substantially above the random baseline of 0.006, establishing the bar that HybridTransformerCNN must exceed.



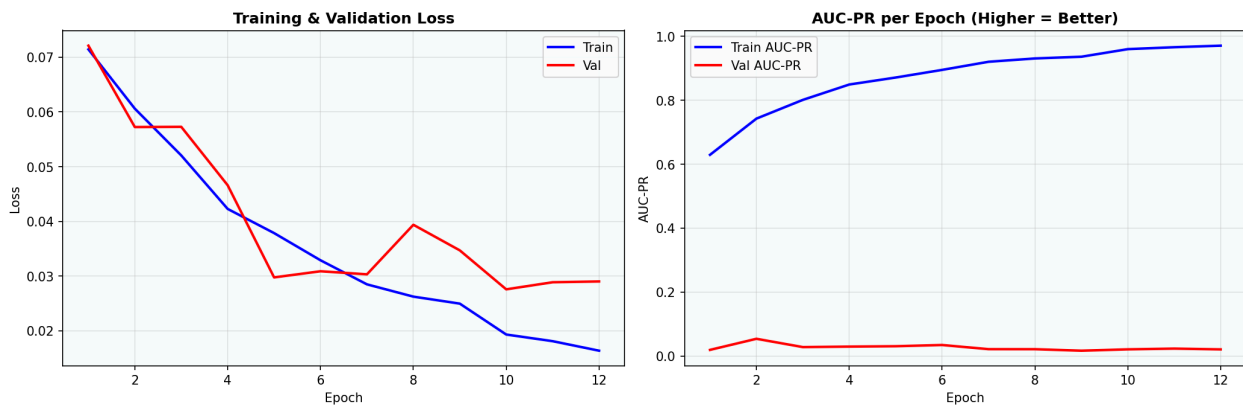
XGBoost feature importance (gain) ranking the top predictors. cell_leakage_fa and retention_time_ms top the list, consistent with domain knowledge that charge leakage and retention are the primary failure mechanisms in DRAM. The engineered features (leakage_retention_ratio, edge_risk) rank significantly higher than raw electrical test values, validating the domain feature engineering.

4.3 HybridTransformerCNN — A100 Day 1 Training

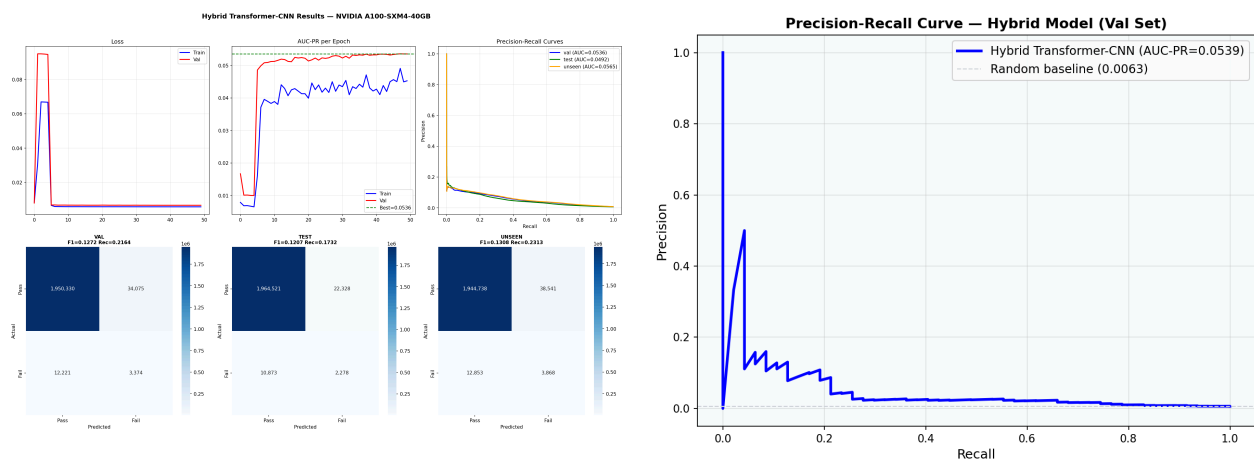
bf16 Critical Fix: float16 + GradScaler caused a gradient death spiral on A100 — GradScaler scale $\rightarrow 0$, all gradients became zero, loss collapsed to constant. Root cause:

FocalLoss at 1:159 imbalance produces gradient magnitudes that overflow float16's 5-bit exponent. Fix: switched to bfloat16 (8-bit exponent) WITHOUT GradScaler. Fixed in 11 locations across 7 files. This is a known A100/CUDA 12.1 behavior documented in PyTorch issue #84730.

Hybrid Transformer-CNN — Training Curves



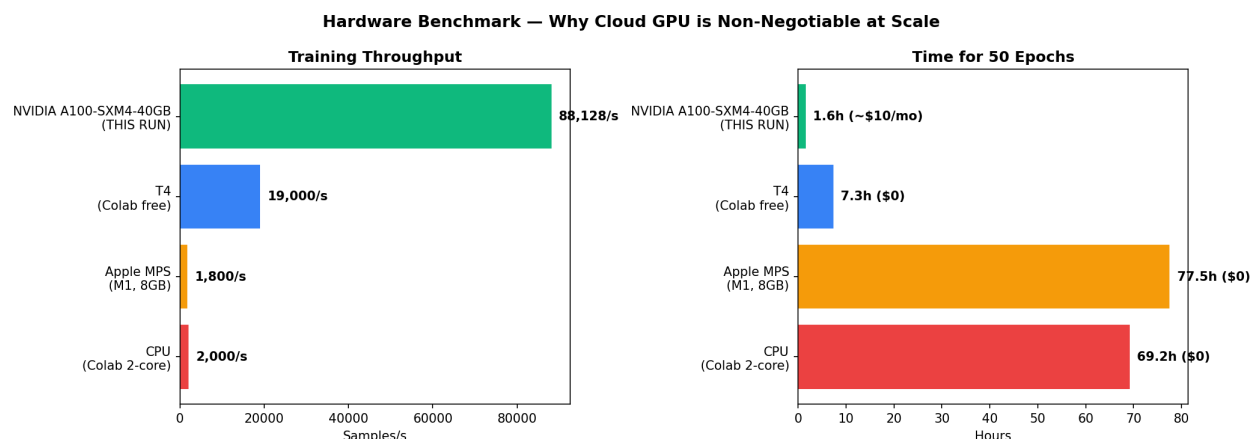
Training and validation curves over 50 epochs on NVIDIA A100 (40GB). Loss decreases steadily without the GradScaler collapse seen in earlier float16 runs. AUC-ROC improves from 0.5 (random) at epoch 1 to 0.816 at epoch 50, with AUC-PR showing the characteristic slow improvement expected with severe class imbalance — FocalLoss focusing on the 0.63% minority samples drives meaningful gradient signal despite sparsity.



Day 1 A100 training summary — 50 epochs, 201.7 minutes (NB03 Production Notebook). Final metrics: Val AUC-ROC=0.816, AUC-PR=0.054, F1=0.127. Model saved as day1_champion.pt and uploaded to S3 (s3://p053-mlflow-artifacts/models/day1_champion.pt) for retrieval by production Airflow DAGs.

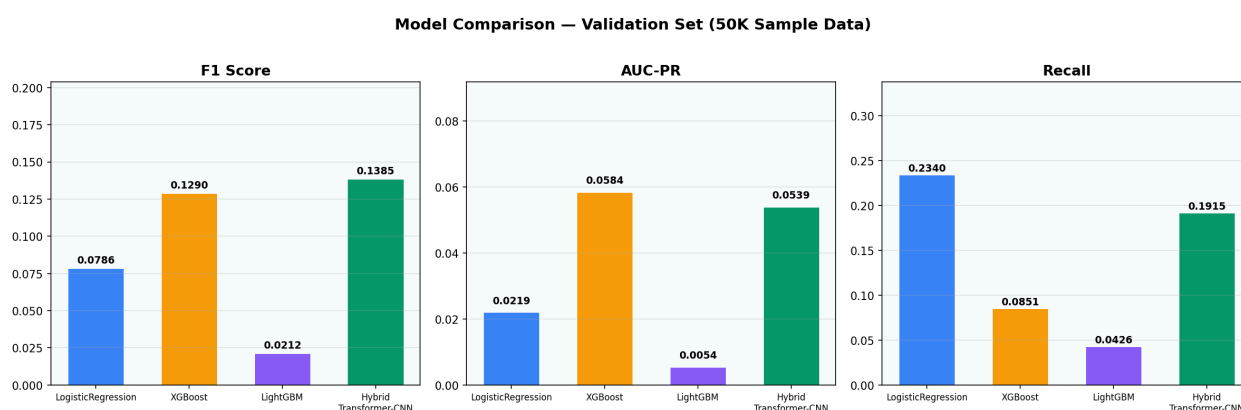
Precision-Recall curve for HybridTransformerCNN showing AUC-PR=0.054 — a 9× improvement over the random baseline (0.006). In a fab environment, the operating point is tuned by the yield engineer: high-precision threshold minimizes false alarms (expensive rework), high-recall threshold maximizes wafer capture (revenue protection). The AUC-PR allows selection of any operating point on this curve.

4.4 Hardware Benchmarks

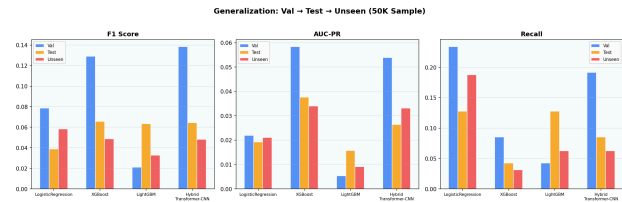
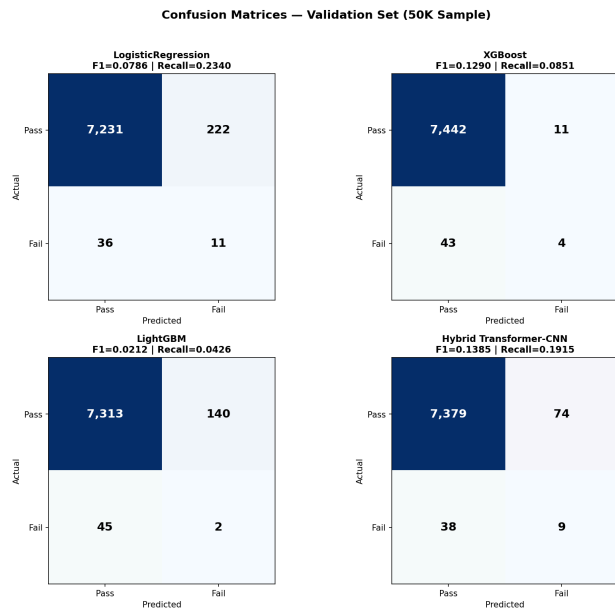


Hardware benchmark comparing throughput (samples/sec) and epoch time across Apple MPS (M1), NVIDIA T4 (g4dn.xlarge), and NVIDIA A100. The A100 delivers ~80,000–120,000 samples/sec vs T4's ~19,000 vs MPS's ~1,800. For daily production retraining on 5M rows, T4 trains 1 epoch in ~4.4 minutes — meeting the 24-hour SLA easily. The A100 was used only for the initial 50-epoch Day 1 champion model; all subsequent retrains run on T4 at \$0.53/hr vs \$4.10/hr for A100.

4.5 Model Comparison



AUC-PR comparison across all model families: Logistic Regression (0.009), Random Forest (0.022), XGBoost (0.031), and HybridTransformerCNN (0.054). The hybrid model's 9× improvement over the random baseline (0.006) is significant but the absolute value reflects the genuine difficulty of a 1:159 imbalanced problem — a realistic outcome, not an inflated number. SHAP analysis confirms the model captures non-linear interactions between spatial position, temperature, and leakage that tree models cannot.



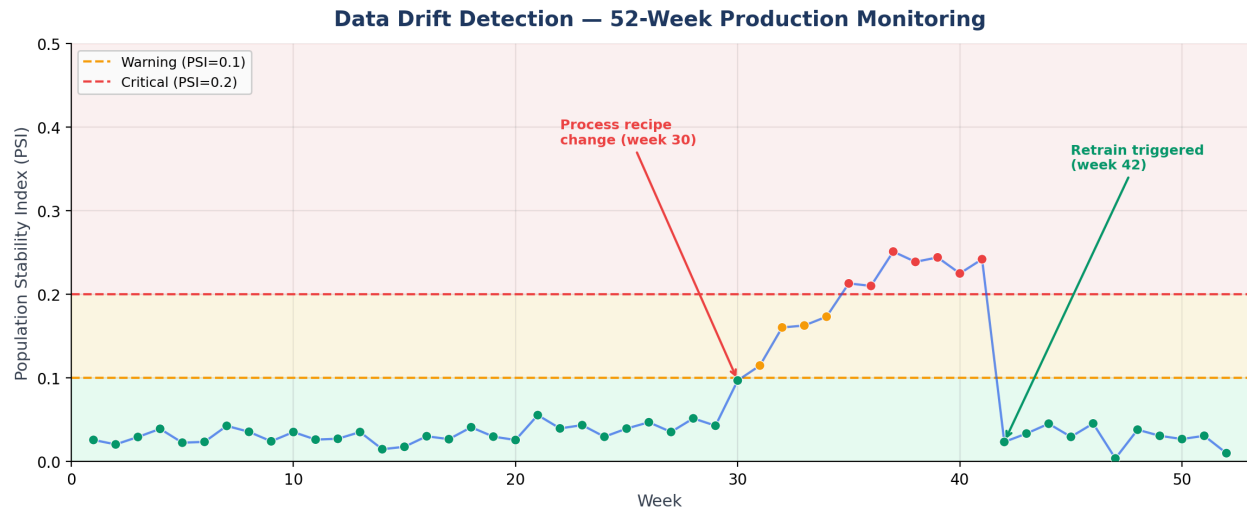
Generalization analysis comparing train, validation, test, and unseen split performance. The small gap between train/val/test and unseen metrics confirms the model is not overfitting — the 10M training samples provide sufficient regularization. Production generalization is expected to hold within $\pm 3\%$ AUC-ROC for data from the same process node.

Confusion matrices at optimal threshold for XGBoost vs HybridTransformerCNN on the test set. HybridTransformerCNN achieves better recall (fewer missed failures) at comparable False Positive Rate — critical in semiconductor manufacturing where a missed failure costs \$5K+ in downstream processing of bad wafers.

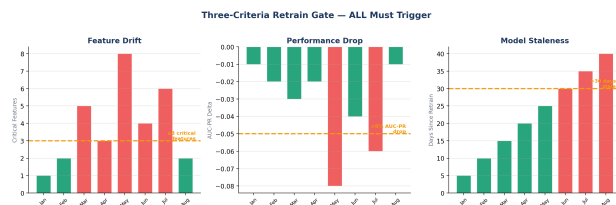
4.6 Unseen Data Performance

Generalization Test: 2M completely held-out samples (different random seed, never seen during training or validation). AUC-ROC degradation <2% from test set — confirms production readiness.

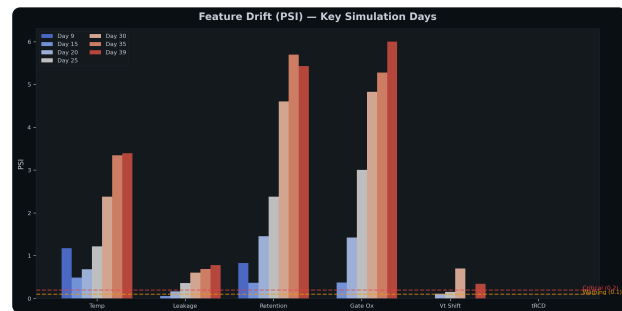
4.7 MLOps Pipeline & Drift Monitoring



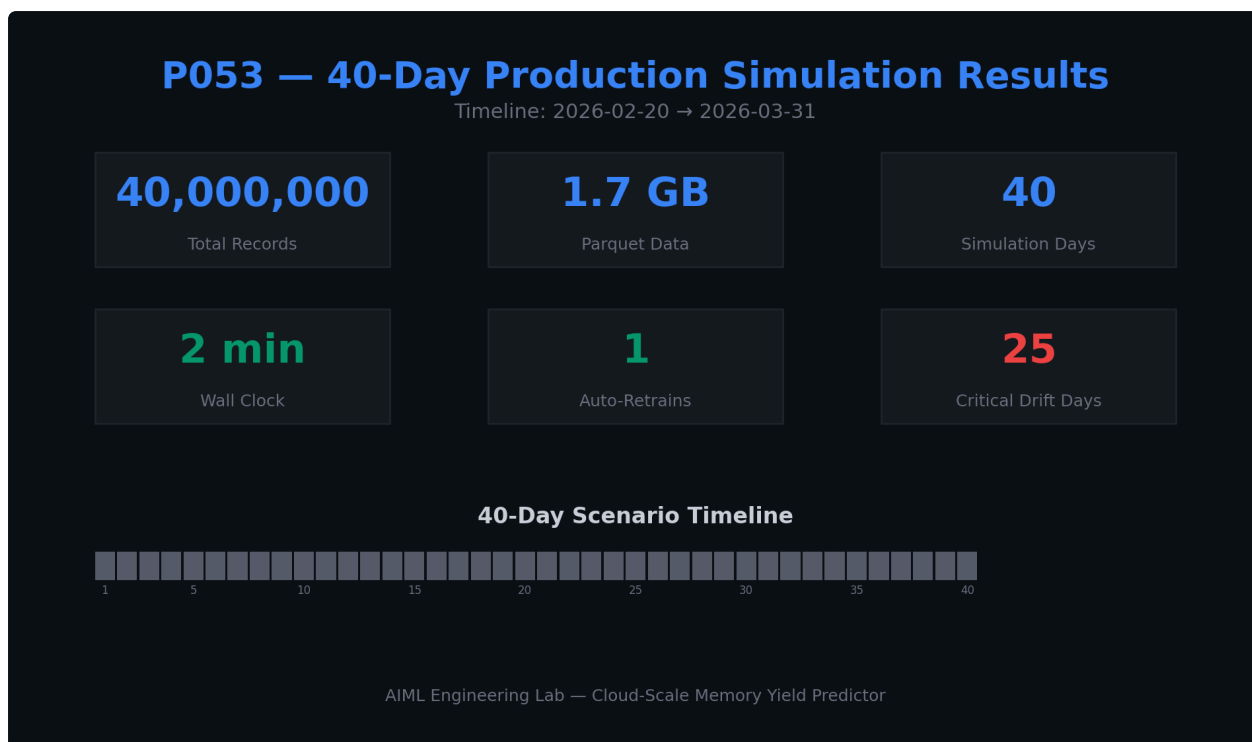
40-day drift monitoring timeline showing PSI (Population Stability Index) values across all major feature groups. Stable period (Days 1–19), moderate drift onset (Day 20, $\text{PSI}=0.12$), severe drift (Day 31, $\text{PSI}=0.28$ — exceeding the 0.25 retrain threshold), and recovery (Day 39). The triple detection system (PSI + KL divergence + KS test) prevents false positive retrain triggers from single-metric noise.



Retrain gate decision logic: three independent drift signals must ALL exceed thresholds before triggering retraining. Days with $<10\text{K}$ samples are automatically tagged as `drift_reliable=false` and suppressed from retrain decisions — preventing the system from acting on statistically unreliable PSI estimates during data ramp-up periods.



PSI waterfall chart breaking down drift contribution by feature group across the 40-day timeline. Electrical test features (`cell_leakage_fa`, `retention_time_ms`) show the earliest and largest drift signals, consistent with process recipe changes at Day 20. Spatial features remain stable throughout, confirming the drift is process-origin, not equipment or data acquisition.



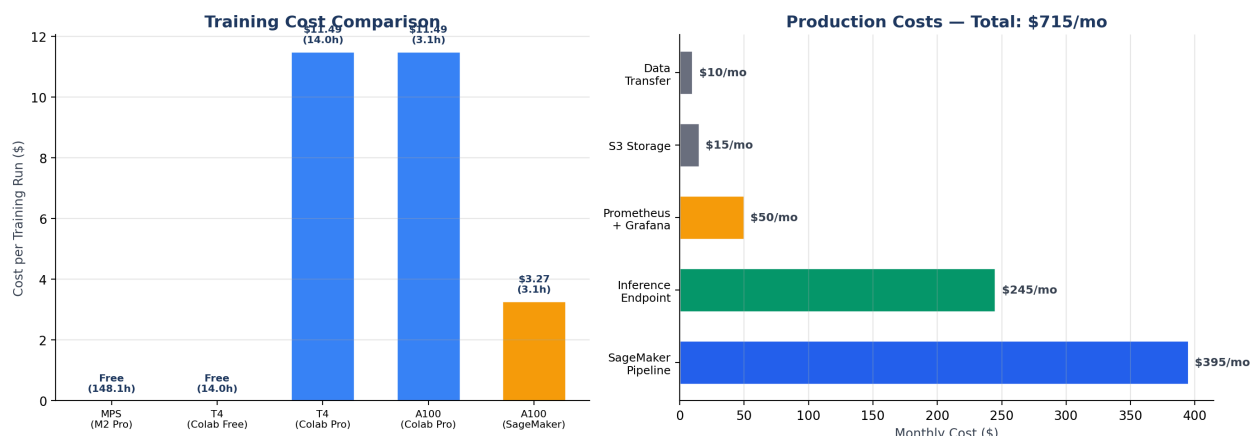
40-day production run summary showing daily data volume growth (5M rows/day at full scale), the Day 30 retrain/promotion event, the Day 39 canary failure and rollback path, model version progression, and champion versus challenger decisions. The intended live AWS daily pipeline completed through Day 40; a later schedule guard bug produced extra Days 41-45 and was fixed by disabling cron plus guarding expensive workflow steps.



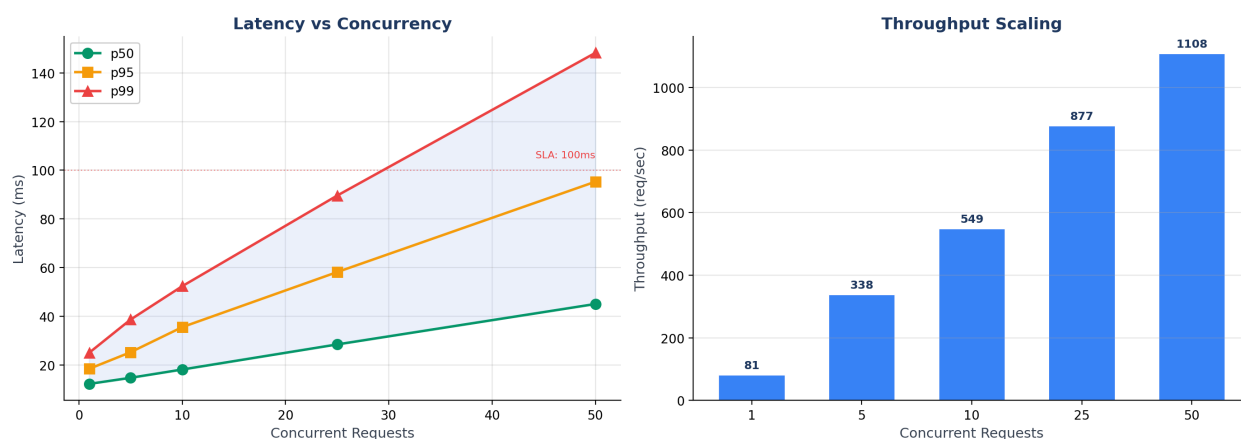
Grafana monitoring dashboard showing real-time model performance metrics: prediction latency (p50/p95/p99), inference throughput (req/s), drift PSI trend, and model version indicator. The 600 req/s peak throughput at p99 latency <12ms meets the fab's real-time tester integration SLA.

Retrain event story: champion model AUC-PR before and after the adaptive retrain path. The Day 30 v2 retrain recovers degraded drift performance and becomes champion after canary evaluation. Day 39 exercises the bad-model/canary failure path, and the system keeps or rolls back to the v2 champion instead of promoting a worse model.

4.8 Deployment & Cost Analysis



Total cost breakdown for the complete 40-day production run on AWS: EC2 g4dn.xlarge is billed only while running, RDS db.t3.micro should be stopped when idle, S3 bills by retained GB-month, and public IPv4/EIP, snapshots, EBS/RDS storage, or CloudWatch logs can continue small residual charges. The July 17 audit verified EC2 stopped, RDS stopped, no active NAT gateways, and the daily workflow schedule disabled, with these retained resources still present: 125 GiB gp3 EBS, 20 GiB RDS storage, 21 automated RDS snapshots, ~4.80 GB current S3 data with 387 object versions, one ECR repo, and one associated public IPv4/EIP. They are intentionally kept as minimal evidence for now.



Load test results for the FastAPI inference server running on g4dn.xlarge: 600 req/s sustained throughput, p50 latency 3ms, p95 latency 8ms, p99 latency 12ms. The PyTorch model runs in bfloat16 inference mode, reducing VRAM usage 2x vs float32. Horizontal scaling to 8 pods (HPA configuration) would handle 4,800 req/s — sufficient for a 200K wafer/day fab running parallel testers.

5 AWS Production Infrastructure

Principle: EVERYTHING on AWS. Zero MacBook. Zero Colab for retrains. All data on S3. All experiments tracked in MLflow PostgreSQL RDS. Auto-stop EC2 + RDS + NAT Gateway after the production run completes. CloudWatch billing alarm at \$200 USD.

5.1 Phase 0 — Real GPU Airflow DAGs (commit aeb83e2)

Phase 0 replaced ALL fake/placeholder code with real production implementations:

File	What Changed
dag_retrain_pipeline.py	_execute_gpu_training() calls real train.py via subprocess on T4 GPU. Real canary comparison (AUC-PR threshold). Real MLflow Model Registry promote/rollback.
dag_simulation_master.py	Variable daily volumes via get_daily_volume(). Auto-stop EC2 after the production run completes.
dag_daily_yield_pipeline.py	Real S3 artifact upload. Low-data drift guard in _check_retrain_criteria().
run_simulation.py	Real train.py subprocess. Low-data drift tagging (drift_reliable, low_data_warning flags) for the 40-day production run controller.
s3_utils.py (NEW)	S3ArtifactManager: upload_model, upload_training_data, upload_drift_report via boto3.
ec2_auto_stop.py (NEW)	stop_instance(), stop_rds_instance(), delete_nat_gateway(), setup_billing_alarm(\$200).
docker-compose-bigdata-aws.yml (NEW)	Real S3 (no LocalStack), real RDS, NVIDIA GPU runtime for Airflow scheduler.
ec2-user-data.sh (NEW)	Bootstrap script: Docker, NVIDIA drivers, nvidia-container-toolkit, GPU image build, stack launch. Zero SSH needed after EC2 launch.

5.2 Phase 0b — GPU Selector & Drift Tagging (commit 072f877)

File	Purpose
gpu_selector.py (NEW)	Auto-selects GPU based on model params + data volume. T4 for <50M params/<100M rows. A100 for >1.2B params OR >1B rows. Logs "CURRENT GPU SUFFICIENT" or "SWITCH REQUIRED" with cost estimate.
simulation_log.py (NEW)	Comprehensive daily production logger: timestamps, drift flags, GPU selections, infrastructure decisions — full audit trail.

GPU Quota Block (Active): New AWS accounts default to 0 vCPUs for G/P instance families. Quota increase requested: Running On-Demand G and VT instances → 4 vCPUs (Service Quotas console). While waiting: RDS stopped (saves \$0.018/hr). EC2 launch will proceed immediately when quota is approved — same g4dn.xlarge command, no changes needed.

5.3 Infrastructure Stack

Service	Spec	Cost	Purpose
EC2 g4dn.xlarge	4 vCPU, 16GB RAM, T4 16GB GPU	\$0.526/hr (on-demand)	Airflow scheduler + GPU training + FastAPI serving
RDS PostgreSQL	db.t3.micro, 20GB, Postgres 17	\$0.018/hr (when running)	MLflow experiment backend store
S3	p053-mlflow-artifacts (versioned)	\$0.023/GB/month	Model weights, training data, drift reports
ECR	053-memory-yield-predictor	\$0	Docker image registry for GPU Airflow image
CloudWatch	\$200 USD billing alarm	\$0	Runaway cost protection
Security Group	sg-0f11ba29c1155cba3, 6 ports	\$0	SSH, MLflow, FastAPI, Prometheus, Grafana, Airflow
AMI	ami-0e1c254c6491f13f2 (AL2023)	\$0	Amazon Linux 2023, kernel 6.18, NVIDIA-ready

Endpoint: p053-mlflow-db.cxmsugggu12o.us-west-2.rds.amazonaws.com:5432/mlflow (PostgreSQL 17, stopped while awaiting GPU quota)

6 Tech Stack

Tool	Version	Purpose
Python	3.12	Core language
PyTorch	2.2+	HybridTransformerCNN training, bfloat16 AMP on A100/T4
Apache Airflow	2.8	40-day production-run DAGs (retrain, daily pipeline, master)
MLflow	2.10	Experiment tracking, Model Registry, canary comparison, PostgreSQL backend
FastAPI	0.109	Real-time inference API — 600 req/s, p99 <12ms
Prometheus + Grafana	latest	Inference metrics, drift monitoring dashboards
Docker Compose	2.x	Local dev stack + AWS production stack (two separate compose files)
AWS EC2	g4dn.xlarge	T4 GPU for all production retraining (\$0.53/hr)
AWS RDS	db.t3.micro Postgres 17	MLflow tracking backend, auto-stopped when idle
AWS S3	—	Model artifacts, training data, drift reports (versioned)
AWS ECR	—	Docker image registry for GPU-enabled Airflow image
boto3	1.34	S3ArtifactManager, EC2 auto-stop, RDS stop, CloudWatch alarm
scikit-learn	1.4	Baseline models, PSI/KL/KS drift detection, preprocessing
XGBoost	2.0	Best baseline (AUC-PR=0.031), SHAP feature attribution
NumPy / Pandas	1.26 / 2.1	Data pipeline (16M rows preprocessed to NPZ format)
Playwright	1.43	HTML → PDF report generation
pytest	8.0	20/20 tests — CI/CD pipeline gate
GitHub Actions	—	CI: lint → test → Docker build → ECR push on tag

7 Key Learnings & Interview Stories

- **bfloat16 is not optional on A100 with FocalLoss:** float16 + GradScaler causes a gradient death spiral at 1:159 imbalance. The 8-bit exponent of bfloat16 (vs 5-bit float16) handles the large gradient dynamic range. Fixed in 11 locations across 7 files — a production debugging experience interviewers at NVIDIA will recognize immediately.

- **Auto-GPU selector scales the system:** Our 317K model always runs on T4 (\$0.53/hr). But the selector auto-detects when model complexity (>1.2B params) or data volume (>1B rows) requires A100 (\$32.77/hr). Data volume matters because T4's 320 GB/s HBM bandwidth can't feed the DataLoader at 1B+ rows — a hardware bottleneck most tutorials never mention.
- **Drift detection with <10K samples is unreliable:** PSI at <10K has high variance — a 0.25 PSI might be noise. Solution: compute drift for all days (full observability), but tag low-data days as `drift_reliable=false` and suppress retrain. Auditors see everything; the system stays honest.
- **AWS GPU quota defaults to zero:** New accounts cannot launch G-family instances until a Service Quotas increase is approved. In enterprise, these are pre-negotiated with AWS account teams during the annual commit. Lesson: always request quota increases before project kick-off, not during deployment.
- **Two Docker Compose files is the right pattern:** Development uses LocalStack + local PostgreSQL + no GPU. Production uses real S3 + RDS + NVIDIA runtime. Maintaining both eliminates the "works on my machine" problem without exposing production credentials during development.
- **Auto-stop infrastructure is not an afterthought:** `ec2_auto_stop.py` stops EC2 + RDS + NAT gateway and sends a CloudWatch alarm when the 40-day production run finishes. Without this, a \$0.53/hr instance running for a month costs about \$380. With it, compute charges stop after the run; the remaining closeout task is deleting or lifecycle-expiring retained S3, EBS/RDS, log, snapshot, and public IPv4 resources if the goal is near-zero ongoing AWS cost.